Aaron Jones

CS 499: Computer Science Capstone — Milestone Three: Enhancement Two, Algorithms and Data Structures

Artifact Description

The artifact enhanced in this milestone is the ABCU Advising Program, a C++ console application originally created for a data structures and algorithms course. The program reads a CSV file of course data — course numbers, titles, and prerequisites — into memory, and gives the user a simple menu to load that data, print an alphanumeric list of every course, or look up a single course's title and prerequisites by its course number. The artifact's original purpose in that course was to compare the runtime and memory trade-offs of different data structures (a vector, a hash table, and a binary search tree) for the same advising task; the version carried forward into this ePortfolio is the vector-based implementation.

Justification for Inclusion and Summary of Improvements

I selected this artifact for the Algorithms and Data Structures category because its original design makes the cost of a data structure choice easy to see and easy to fix. The program's single most frequent operation — looking up one course by its course number — was implemented as a linear scan through a vector using find_if, checking courses one at a time until it found a match or reached the end. That is an O(n) operation: the more courses in the file, the longer every single lookup takes, even though a course number is exactly the kind of unique key a hash-based structure is built for.

The core enhancement replaces the vector of courses with an unordered_map keyed by course number. Looking up a course by number now goes through courseData.find(), an O(1) average-case hash lookup, instead of scanning every course in the file one at a time. Printing the alphanumeric course list still needs the courses in sorted order, so that path pulls the values out of the hash map into a vector and sorts them, but only at the moment a listing is actually requested — the same point in the program where the original did its sorting. The practical difference shows up specifically in lookups: with the original vector, finding course CS-300 in a 50-course file could mean checking up to 50 courses one at a time; with the hash map, that same lookup is a near-constant-time operation regardless of how many courses are loaded.

Beyond the planned data-structure swap, testing the enhanced program against the real course-data file surfaced four additional issues I fixed along the way. First, loadDataStructure() never checked whether the file actually opened; giving it a misspelled filename silently loaded zero courses while still printing a success message, which I fixed by checking the stream and reporting a real error. Second, and most significant: the real input file uses Windows-style (CRLF) line endings, and C++'s getline() only strips the trailing newline, leaving a stray carriage-return character attached to the last field on every line — either the course title or its final prerequisite — which corrupted stored data in a way that produced no visible error and was easy to miss entirely. I fixed this by stripping a trailing carriage return from each line before parsing it. Third, the original stored every parsed line even when it was missing a course number or title, silently keeping malformed records; the enhanced version validates each row

and reports and skips a malformed one instead. Fourth, typing anything non-numeric at the menu prompt left cin in a failed state that was never cleared, which caused every subsequent read to fail immediately without consuming input — an infinite loop of "Invalid selection" messages with no way to recover short of restarting the program. I fixed this by clearing the stream's error state and discarding the bad input before looping again.

Course Outcomes

This enhancement most directly demonstrates Outcome 3, designing and evaluating a computing solution using algorithmic principles while managing the trade-offs involved in a design choice, and Outcome 4, using well-founded tools and techniques to deliver a working improvement. Outcome 3 is demonstrated in the data-structure swap itself: moving from a vector to a hash map is a genuine trade-off rather than a strictly better choice in every respect — the program gains near-constant-time lookups but gives up the vector's inherent ordering, which is why the alphanumeric listing now has to rebuild and sort a temporary vector from the hash map's contents instead of iterating over an already-ordered structure. Outcome 4 is demonstrated by the fact that this is working, tested code rather than a description of an improvement: the program compiles cleanly, runs correctly against the real course-data file, and the four bugs found along the way are each fixed and verified, not just identified.

Reflection on the Enhancement Process

The most useful lesson from this enhancement was realizing how much of the real risk was hiding in the data file itself rather than in the algorithm. The vector-to-hash-map swap was conceptually the easy part; the carriage-return bug was the one that actually mattered, because it produced no crash and no error message, just quietly corrupted the last field on every line of a Windows-formatted file. If I had only tested with a file I'd typed by hand in a Linux-style editor, I likely would never have seen it at all. That was a good reminder that correctness testing has to include realistic data, not just a hand-crafted test case that happens to avoid the conditions a real file will actually have.

The infinite-loop bug was a similar kind of lesson from a different angle: it was never part of the original Module One enhancement plan at all, since that plan was about the data structure, not the input loop. I found it only because fixing the data structure meant touching the surrounding menu code closely enough to notice that an invalid choice didn't just print a warning once — it would print it forever. That pushed me, again, to decide how far to go past the specific scope I had planned; I fixed it because it was a real, user-facing defect in code I was already working in, and documented it as a distinct fix separate from the core enhancement.

One limitation I want to be direct about rather than quietly leave out: the program reads a filename for option 1 using cin >>, which stops reading at the first whitespace character. A filename containing a space will silently fail to open rather than being read correctly. I ran into this directly while testing on my own machine and worked around it by renaming the file to remove the space, rather than changing the input-handling code, since it was outside the scope of this milestone's enhancement. It's a real, known limitation of the artifact as it stands today, and a genuine candidate for a future enhancement — reading the whole line with getline() instead of cin >> — rather than something this milestone's work already addresses.

AI Usage Acknowledgment: Generative AI (Claude) was used to help implement the planned data-structure enhancement, identify and fix related bugs, test the implementation against real course data, and draft language for this narrative. The decisions about enhancement scope and all final wording are my own.